23CSE202 DATABASE MANAGEMENT SYSTEMS

Department of Computer Science and Engineering | Amrita School of Computing

PROJECT REPORT: QuickBite! Online Food Ordering System

1. Abstract

The QuickBite! Online Food Ordering System is a full-stack, database-centric web application engineered to solve critical inefficiencies in traditional food ordering, such as manual calculation errors, lack of delivery tracking, and missing transaction guarantees. The core is a 3NF-normalized relational database comprising 19 tables in SQLite with strict declarative constraints (CHECK, UNIQUE, NOT NULL), cascading referential integrity, and ACID transaction boundaries (BEGIN TRANSACTION ... COMMIT / ROLLBACK). It supports four roles: Customer, Restaurant Admin, Delivery Driver, and Platform Admin. Key database features include 4 analytical views, 3 automatic triggers for rating calculations and order timestamp updates, 8 B-Tree indexes, item variants/addons, promo coupons, temporal price freezing (price_at_order_time), and an interactive ACID transaction lab with EXPLAIN QUERY PLAN inspection.

2. Introduction

In traditional food ordering (phone calls, counter tickets, or paper receipts), restaurants suffer from communication errors, missing receipts, and zero customer tracking. Furthermore, changing food prices retrospectively corrupts past order bills if joined dynamically without temporal price freezing. QuickBite! provides a centralized, normalized database system that automates ordering with atomic ACID guarantees, preserves historical order prices at purchase time, updates restaurant ratings via triggers, and provides analytical views for sales summaries, busiest hours, and popular cuisines.

3. Requirements (Functional & Non-Functional)

• Functional Requirements:\n- Public/Guest: User registration, bcrypt password hashing, JWT authentication, and Google OAuth 2.0 login.\n- Customer: Browse restaurants, cuisine filtering, item variants (sizes) and add-ons (toppings), persistent cart, coupon code validation, ACID checkout, animated delivery tracking, order history search, itemized tax invoice PDF, and ratings/reviews.\n- Restaurant Admin: Live kitchen queue management, status updates (Preparing, Ready), and menu CRUD operations.\n- Delivery Driver: Courier dispatch dashboard and status transition controls (Picked Up, Delivered).\n- Platform Admin: Platform revenue metrics, interactive 10-query SQL runner, query execution plan viewer, ACID rollback simulator, and one-click database reset.\n\n• Non-Functional Requirements:\n- Security: Passwords hashed with bcrypt (salt cost: 10); stateless JWT auth tokens; role-based access control.\n- Performance: 8 B-Tree indexes on search and foreign key columns; in-memory query execution via SQLite WASM.\n- Reliability: PRAGMA foreign_keys = ON; ACID atomicity on checkout ensuring zero orphan records on payment failure.\n- Usability: Responsive dark theme via Tailwind CSS; native Web Audio API sound effects; 4s live polling.

4. UI Design

The application provides responsive screens for all user roles:\n1. Home Page (/): Restaurant directory, horizontal cuisine selector pills, search bar, and delivery ETA cards.\n2. Restaurant Details (/restaurant/:id): Categorized dishes, veg toggle, 3D card tilt effect, rating breakdown bars, and review star filters.\n3. Item Customization Modal: Select sizes (variants) and extra toppings (addons) with dynamic price calculation.\n4. Checkout Page (/checkout): Address input, promo code chips, payment selection (Card/UPI/COD), and bill calculation.\n5. Order Tracking (/tracking/:id): 5-step status stepper, animated SVG courier route map, and invoice button.\n6. Order History (/orders): Chronological order history with item/restaurant search and status filter pills.\n7. Tax Invoice Modal: Itemized bill breakdown, promo discount, tax, delivery fee, and print trigger.\n8. Restaurant Admin (/restaurant-admin): Kitchen queue and menu CRUD table.\n9. SQL Showcase & ACID Lab (/admin): Viva demo runner for 10 queries, execution timing, EXPLAIN plans, and ACID rollback simulation.

5. Database Design

• Relational Schema (19 Tables):\nusers, addresses, cuisines, restaurants, restaurant_cuisines, menu_categories, menu_items, menu_item_variants, menu_item_addons, coupons, carts, cart_items, orders, order_items, payments, delivery_personnel, delivery_assignments, reviews, favorites.\n\n• Normalization (1NF, 2NF, 3NF/BCNF):\n- 1NF: All attributes are atomic; multi-valued items and cuisines are decomposed into junction tables.\n- 2NF: No partial dependencies on composite keys (restaurant_cuisines and favorites).\n- 3NF/BCNF: No transitive functional dependencies exist. Non-key attributes depend strictly on candidate keys. The price_at_order_time column in order_items preserves the historical purchase price at the exact moment of order placement without violating 3NF.

6. Database Selection & Justification

SQLite (via sql.js WebAssembly engine) was selected because:\n1. Full ACID Compliance: Implements atomic commits

and rollback journals for multi-table transactions.\n2. Relational Integrity: Enforces foreign key constraints (ON DELETE CASCADE / RESTRICT), CHECK constraints, and triggers.\n3. Query Plan Inspection: Supports EXPLAIN QUERY PLAN for verifying B-Tree index utilization during viva evaluation.\n4. Self-Contained Portability: Operates from a single binary file (database/food_delivery.db) requiring zero external DBMS daemon setup.

7. Implementation & Key SQL

• ACID Checkout Transaction:\nWrapping order header, order items, payment transaction, and cart clearing between BEGIN TRANSACTION and COMMIT/ROLLBACK ensures atomic operations.\n\n• Analytical Views: \nv_restaurant_sales_summary, v_top_rated_menu_items, v_busiest_hours, v_popular_cuisines.\n\n• Database Triggers: \ntrg_review_insert_rating (recalculates avg_rating on reviews) and trg_order_update_timestamp.

8. Testing & Validation

The system was tested against 12 validation test cases:\n- TC-01: Duplicate email rejected (HTTP 400) -> PASS\n- TC-02: Invalid password rejected (HTTP 401) -> PASS\n- TC-03: Valid user authentication returning JWT -> PASS\n- TC-04: Role escalation blocked by requireRole (HTTP 403) -> PASS\n- TC-05: Negative price CHECK constraint violation rejected by SQLite -> PASS\n- TC-06: Out-of-range rating (>5) CHECK constraint rejected -> PASS\n- TC-07: Empty cart checkout rejected (HTTP 400) -> PASS\n- TC-08: Promo code QUICKBITE20 applies 20% discount correctly -> PASS\n- TC-09: ACID Rollback simulation verifies 0 orphan rows created on payment failure -> PASS\n- TC-10: ACID Commit creates order, items, and payment atomically -> PASS\n- TC-11: Rating trigger updates restaurant avg_rating on review insert -> PASS\n- TC-12: Reset DB endpoint drops and re-seeds database cleanly -> PASS

9. Results & Screenshots

Key modules captured for evaluation:\n- Fig 9.1: Home Page restaurant directory with cuisine filter pills and search bar.\n- Fig 9.2: Restaurant menu with 3D card tilt and rating breakdown progress bars.\n- Fig 9.3: Item Customization Modal showing size variants and topping add-ons.\n- Fig 9.4: Checkout Page with promo code quick-apply chips and live total calculation.\n- Fig 9.5: Order Tracking Page with animated SVG courier route map and 5-stage stepper.\n- Fig 9.6: Tax Invoice Modal showing printable itemized bill and transaction reference.\n- Fig 9.7: Order History Page with search bar and status filter pills.\n- Fig 9.8: SQL Showcase Tab 1 with query runner, execution time, and EXPLAIN plan.\n- Fig 9.9: SQL Showcase Tab 2 with ACID rollback audit table showing net changes.

10. Challenges Faced & Solutions

1. WebAssembly SQLite Concurrency: Calling db.export() during open transactions caused WAL locks. Solved by introducing an inTransaction flag in db.js that delays disk sync until COMMIT or ROLLBACK.\n2. Temporal Price Consistency: Changing menu prices altered past order totals. Solved by storing price_at_order_time in order_items.\n3. Rating Calculation Overhead: Recomputing AVG(rating) on every card load was slow. Solved by implementing automatic database triggers caching avg_rating.\n4. Audio Latency: External audio CDNs caused CORS issues. Solved by building a native Web Audio API synthesizer with zero external dependencies.

11. Conclusion & Future Enhancements

The QuickBite! project successfully fulfills all 23CSE202 DBMS course requirements, demonstrating 19 normalized 3NF tables, full ACID transaction boundaries, automated triggers, analytical views, and role-based workflows.\nFuture Enhancements: (1) PostGIS / SpatiaLite for GPS distance calculation, (2) WebSockets for live driver location push, and (3) live Stripe/Razorpay payment gateway integration.

References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill.\n2. SQLite Query Planner Documentation: https://www.sqlite.org/eqp.html\n3. sql.js Documentation: https://github.com/sql-js/sql.js\n4. Express.js Framework: https://expressjs.com/\n5. React 18 TypeScript: https://react.dev/\n6. Tailwind CSS: https://tailwindcss.com/